

FAQ Assistant API

Project Development Phase Document

Item	Details
Project Name	FAQ Assistant API
Document	Development Phase
Version	1.0 (Draft)
Phase	Phase 4 - Development
Status	For review

1. Introduction

This document describes how the FAQ Assistant API will be built during the development phase. It covers the development approach, environment setup, project structure, modules, API endpoints, database, coding standards and the schedule of tasks. It follows the planning, requirements and design phases.

2. Development Objectives

- Implement all high-priority features defined in the planning document.
- Build a clean, modular and well-documented codebase.
- Deliver a working API that answers questions with a relevance score.
- Write unit tests alongside the code so bugs are caught early.
- Prepare a stable build that can move to the testing phase.

3. Development Approach

The team will use an iterative (Agile-style) approach with short sprints of one week each. Each sprint delivers a small working part of the API that is reviewed before the next sprint starts.

- Version control with Git; every feature is built in its own branch and merged by pull request.
- Code review by another team member before merging.
- Tasks tracked on a simple board (To Do, In Progress, Review, Done).
- Short weekly review to check progress and adjust the plan.

4. Development Environment

Item	Tool / Version
Language	Python 3.10+ (or chosen language from the planning phase)
Framework	FastAPI with Uvicorn server
Database	PostgreSQL (SQLite for local development)
ORM / Migrations	SQLAlchemy and Alembic
Matching library	scikit-learn (TF-IDF) or sentence-transformers
Testing	Pytest and Postman
Code editor	VS Code
Version control	Git and GitHub
Containers	Docker (for consistent setup and deployment)

4.1 Setup Steps

- Clone the repository and create a virtual environment.
- Install dependencies from requirements.txt.

- Copy .env.example to .env and set database URL and secret keys.
- Run database migrations to create the tables.
- Start the server and open the Swagger page at /docs to verify.

5. Project Structure

```
faq-assistant-api/
|-- app/
|   |-- main.py           # application entry point
|   |-- config.py         # settings and environment variables
|   |-- api/
|       |-- ask.py        # question-answer endpoint
|       |-- faqs.py       # FAQ management endpoints
|       |-- auth.py       # authentication routes
|   |-- models/           # database models
|   |-- schemas/          # request and response schemas
|   |-- services/
|       |-- matcher.py    # question matching logic
|       |-- logger.py     # query logging
|   |-- core/             # security, rate limiting, error handlers
|-- tests/                # unit and integration tests
|-- migrations/           # database migration files
|-- requirements.txt
|-- Dockerfile
|-- README.md
```

6. Module Breakdown

Module	Purpose	Key Tasks
Configuration	Load settings safely	Environment variables, config classes
Database layer	Store FAQs, categories and logs	Models, migrations, CRUD functions
FAQ Management	Admin control of content	Create, update, delete, list, search FAQs
Matching Engine	Find best answer for a question	Text cleaning, similarity scoring, threshold check
Ask Endpoint	Serve answers to clients	Validate input, call matcher, format response
Authentication	Protect admin actions	API key / JWT generation and verification
Logging and Feedback	Learn from usage	Save queries, unanswered questions, feedback
Error Handling	Consistent responses	Custom exceptions and standard error format

7. API Endpoints

Method	Endpoint	Description	Access
POST	/ask	Submit a question and get the best answer	Public / API key
GET	/faqs	List FAQs with filters and pagination	Admin
GET	/faqs/{id}	Get one FAQ	Admin

Method	Endpoint	Description	Access
POST	/faqs	Create a new FAQ	Admin
PUT	/faqs/{id}	Update an FAQ	Admin
DELETE	/faqs/{id}	Delete an FAQ	Admin
POST	/feedback	Mark an answer helpful or not helpful	Public / API key
GET	/unanswered	List questions with no good match	Admin
GET	/health	Check that the service is running	Public

7.1 Sample Request and Response

```

POST /ask
{
  "question": "How can I reset my password?"
}

200 OK
{
  "answer": "Click 'Forgot password' on the login page ...",
  "matched_question": "How do I reset my password?",
  "confidence": 0.91,
  "faq_id": 12
}

```

8. Database Design (Summary)

Table	Main Fields
faqs	id, question, answer, category_id, tags, is_active, created_at, updated_at
categories	id, name, description
query_logs	id, user_question, matched_faq_id, confidence, created_at
feedback	id, query_log_id, is_helpful, comment, created_at
api_keys	id, key_hash, owner, role, is_active, created_at

9. Matching Logic

The matching engine decides which FAQ best answers the user's question. It works in these steps:

- Clean the question: lowercase, remove punctuation and extra spaces, remove stop words.
- Convert the question and all stored FAQ questions into vectors (TF-IDF or embeddings).
- Calculate cosine similarity between the user question and each FAQ.
- Pick the FAQ with the highest score.
- If the score is above the threshold (for example 0.60), return the answer with the score.
- If the score is below the threshold, return a fallback message and log the question as unanswered.

Vectors of FAQs are calculated once and refreshed whenever an FAQ is added, edited or deleted, so each request is fast.

10. Security Implementation

- Admin endpoints require a valid token; only hashed API keys are stored.
- All inputs are validated with schemas to prevent bad data and injection.
- Secrets are kept in environment variables, never in the code.
- Rate limiting on public endpoints to prevent misuse.
- HTTPS enabled in deployment and CORS restricted to allowed origins.

11. Error Handling Standard

Code	Meaning	Example
200	Success	Answer returned
201	Created	New FAQ added
400	Bad request	Empty or too long question
401 / 403	Unauthorized / Forbidden	Missing or invalid token
404	Not found	FAQ id does not exist
429	Too many requests	Rate limit exceeded
500	Server error	Unexpected failure (logged)

12. Coding Standards

- Follow PEP 8 style and use a formatter (Black) and linter (Flake8).
- Use meaningful names for files, functions and variables.
- Add docstrings and type hints for public functions.
- Keep functions small and single-purpose; avoid duplicate code.
- Never commit passwords, keys or .env files.
- Use clear commit messages, for example: feat: add FAQ create endpoint.

13. Sprint Plan

Sprint	Tasks	Output
Sprint 1 (Week 1)	Project setup, database models, migrations, config, health endpoint	Running skeleton with database
Sprint 2 (Week 2)	FAQ CRUD endpoints, categories, authentication	Admin can manage FAQs securely
Sprint 3 (Week 3)	Matching engine, /ask endpoint, fallback handling	Working question-answer flow
Sprint 4 (Week 4)	Logging, feedback, rate limiting, Swagger docs, unit tests, bug fixes	Stable build ready for testing

14. Testing During Development

- Unit tests for the matching logic, schemas and helper functions.
- API tests for every endpoint with valid and invalid inputs.
- Minimum target of around 70 percent code coverage before handing over to QA.
- Tests run automatically on every pull request (CI).

15. Deliverables

- Complete source code in the Git repository.
- Database schema and migration files.
- Postman collection and auto-generated Swagger documentation.
- README with setup and run instructions.
- Unit test suite and coverage report.

16. Development Risks

Risk	Mitigation
Low matching accuracy	Tune threshold, try embeddings, test with real questions
Delay in a sprint	Move low-priority tasks to next sprint, keep buffer time
Integration issues between modules	Agree on schemas early, write integration tests
Code conflicts in team work	Small branches, frequent merges, code reviews

17. Exit Criteria for This Phase

- All high-priority endpoints are implemented and working.
- Unit tests pass and code review is complete.
- Documentation and README are updated.
- Build is deployed to a test environment for the testing phase.